Reviewer Pack — Tropical Representation Theory and AC0 Parity Circuit Comple...

Entry cedd2b57040e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 17:23:23 UTC

Tropical Representation Theory and AC0 Parity Circuit Complexity

Entry ID: `cedd2b57040e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 17:23:23 UTC

1. Conjecture

Field A (mathematical branch): Tropical Representation Theory **Field B** (complexity object): AC0 Parity Circuit Complexity

Statement:

> ['For any AC0 circuit C computing PARITY on n inputs, the number of distinct tropical representations of its output with respect to the action of the general linear group over the finite field F_2 is at least Ω(n^{1/3}).', "Equivalently, the maximum order of a monomial in the tropical polynomial representation of C's output over F_2 is at least Ω(n^{1/3})."]

Rationale (proposer's reasoning):

> ['Tropical representation theory provides a combinatorial tool to study algebraic objects. If this conjecture holds, it would suggest that AC0 parity circuits have a rich tropical structure, which could be exploited to prove lower bounds on their size.', 'This might offer a new perspective on the complexity of PARITY and potentially lead to non-relativizing proof techniques.']

Taxonomy category: `LIFTING` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `32cc95cf0a22aa5d`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> To support the conjecture, the average number of distinct tropical representations and the average maximum order of a monomial must both be at least Ω(n^{1/3}) across all AC0 circuits with n ≤ 40, for each circuit tested with 30 random seeds.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `tropical representation theory AND AC0 parity circuit complexity` - `PARITY circuit complexity IN tropical representation theory` - `general linear group action ON AC0 circuits WITH tropical polynomial`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_ac0_circuit(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def tropical_representation(circuit):
        n = len(circuit)
        if n == 1:
            return {circuit[0]: Fraction(1)}
        else:
            left_rep = tropical_representation(circuit[:n//2])
            right_rep = tropical_representation(circuit[n//2:])
            rep = {}
            for a in left_rep:
                for b in right_rep:
                    if a + b not in rep or left_rep[a] * right_rep[b] > rep[a + b]:
                        rep[a + b] = left_rep[a] * right_rep[b]
            return rep

    def max_order(rep):
        return max(rep.values()) if rep else 0

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        circuit = generate_ac0_circuit(n)
        rep = tropical_representation(circuit)
```

```python
        distinct_representations = len(rep)
        max_order_value = max_order(rep)
        results.append(max_order_value)

        if distinct_representations < math.ceil(n ** (1/3)) or max_order_value < math.ceil(n ** (1
            return {
                "metric_name": "Distinct Tropical Representations and Max Order",
                "metric_value": 0,
                "instances_tested": n_values[-1],
                "conjecture_holds": False,
                "counterexample": f"n={n}, distinct_representations={distinct_representations}, ma
            }

    mean = sum(results) / len(results)
    std_dev = math.sqrt(sum((x - mean) ** 2 for x in results) / len(results))
    support_fraction = sum(1 for r in results if r >= math.ceil(n_values[-1] ** (1/3))) / len(resu

    return {
        "metric_name": "Distinct Tropical Representations and Max Order",
        "metric_value": mean,
        "instances_tested": n_values[-1],
        "conjecture_holds": support_fraction == 1.0,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71,
        73, 79, 83, 89, 97, 101, 103, 107, 109, 113
    ]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result["metric_value"])

    mean = sum(results) / len(results)
    std_dev = math.sqrt(sum((x - mean) ** 2 for x in results) / len(results))
    support_fraction = sum(1 for r in results if r >= math.ceil(seeds[-1] ** (1/3))) / len(results

    if support_fraction == 1.0:
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    elif support_fraction > 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(i for i, r in enumerate(results) if r < math.ceil(seeds[-1] ** (
        print(f"RESULT: FALSIFIED counterexample='n={seeds[first_failing_seed]}, distinct_represen
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)


```
etric_name': 'Distinct Tropical Representations and Max Order', 'metric_value': 0, 'instances_tested'
TRIAL: {'metric_name': 'Distinct Tropical Representations and Max Order', 'metric_value': 0, 'instanc
TRIAL: {'metric_name': 'Distinct Tropical Representations and Max Order', 'metric_value': 0, 'instanc
TRIAL: {'metric_name': 'Distinct Tropical Representations and Max Order', 'metric_value': 0, 'instanc
TRIAL: {'metric_name': 'Distinct Tropical Representations and Max Order', 'metric_value': 0, 'instanc
TRIAL: {'metric_name': 'Distinct Tropical Representations and Max Order', 'metric_value': 0, 'instanc
TRIAL: {'metric_name': 'Distinct Tropical Representations and Max Order', 'metric_value': 0, 'instanc
TRIAL: {'metric_name': 'Distinct Tropical Representations and Max Order', 'metric_value': 0, 'instanc
TRIAL: {'metric_name': 'Distinct Tropical Representations and Max Order', 'metric_value': 0, 'instanc
TRIAL: {'metric_name': 'Distinct Tropical Representations and Max Order', 'metric_value': 0, 'instanc
```


8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> The test only considered up to n=11, which is too small to establish the conjecture's validity for larger values of n. The metric does not scale trivially with n, but a counterexample at n=11 does not necessarily invalidate the conjecture for all n.

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Safety rail: critic_challenge_falsified | original: The test results show that for n=11, there was only one distinct tropical representation and the maximum order of a monomial was zero, which does not | next: Further investigation is needed to determine if the conjecture holds for all AC0 circuits. Additional tests with larger values of n and more varied seeds are required to confirm or refute the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14226
2	preregistration	ollama_remote	glm4:latest	0	0	9668
3	novelty	ollama_remote	glm4:latest	0	0	8181
4	novelty	ollama_remote	glm4:latest	0	0	9286
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13660
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9558
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9894
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11079
9	critic	ollama_remote	glm4:latest	0	0	12367
10	judge	ollama_remote	glm4:latest	0	0	9789

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 107707 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/cedd2b57040e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/cedd2b57040e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/cedd2b57040e.tar.gz` (if generated)